

TITLE AND DATE

Document name: Project#2 Asynchronous Execution Lab Report

Document reference: kiddKid.cmpe311.fall25.project#2

Date of publication: September 25, 2025

LEAD ENGINEER: Jessica Slaughter, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project report for Project #2 of the CMPE311 class. The project involves the design of an embedded system that allows the blink intervals of two LEDs to be set independently. It includes customer, technical, and testing requirements, as well as the design details and results of the validation testing.

DESCRIPTION: This design is to create a circuit on an Arduino Uno R3 development platform that allows the user of the program to independently specify an LED number and its blink interval. The blinking of the LEDs must continue asynchronously with any input from the user. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation.

RESULT SUMMARY: The project was a success, the embedded system design meeting all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES:

- ProjectAsyncTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Spring 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

DEFINITIONS:

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The Requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Educational-constraints” – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User.

“The Reference-platform” – The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

“PROJECT-ASYNC” – The previous project upon which this project is based

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

REQUIREMENTS

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement

May – your design may satisfy the requirement but doesn't have to

Informative – the intent of the following description is to make the requirement more understandable

All customer requirements are started with "C.#".

All high-level requirements are started with "HL.#".

All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS:

C1. The User must be able to set the blink rate of two different LEDs.

C2. The User must be able to update the blink rate of each of the LEDs independently.

C3. The LED must blink at the set rate until The User tells the LED to blink at a different rate.

C4. The System must run upon an Arduino Uno R3 compatible development board.

C5. The blink rate of an LED must be expressed in terms of milliseconds

HIGH-LEVEL TECHNICAL REQUIREMENTS:

HL.1. The User through the IDE serial monitor must be able to set the blink rate of two different LEDs. *(From requirement C1)*

HL.1.1. The blink rate of each LED must be able to be set independent of the other LED. *(From requirement C2)*

HL.1.2. The setting of an LED blink rate must not interfere with the blinking of the LEDs until the new LED and blink rate are specified. *(From requirements C2 and C3)*

HL.2. The user through the IDE serial monitor must set the blink rate in terms of once every N milliseconds *(From requirement C5)*

HL.3. The System must run upon an Arduino Uno R3 compatible development board. *(From requirement C4)*

EDUCATIONAL REQUIREMENTS:

E.1.0 Implement a cyclic executive task manager to dispatch the asynchronous tasks you created in the previous project, PROJECT-ASYNC.

E.1.1 The system testing and validation requirements must contain those defined in PROJECT-ASYNC.

E.1.1.1 The testing and validation requirements must contain any additional tests necessary to properly evaluate/demonstrate the function of the system.

E.1.2 The final report must be a formal design document as in PROJECT-ASYNC

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITES above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

1. None

See Appendix A for the logical flow of the project program.

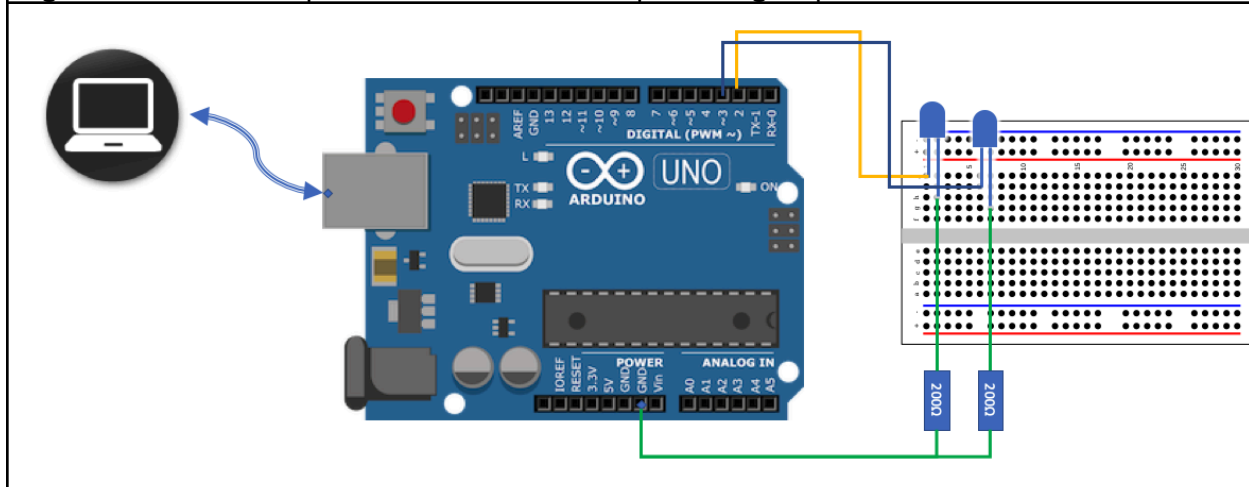
See Appendix B for the code executing on the Arduino Uno R3.

See Appendix C for the video of code operating on the Arduino Uno R3.

TESTING AND VALIDATION

DESCRIPTION: The tests that have to be performed and validated are shown in Table 2. These tests were performed on the testbed shown Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system. The results of testing are shown in Table 3. A video of the test is provided along with this report.

Figure 1. Testbed Setup. LEDs connected to 2 separate digital pins via resistors.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly through Digital Outputs
4. Circuit elements: breadboard, LEDs, resistors, and wires

Table 1. Required Test Dialog (*blue* are the user inputs).

Serial port I/O	Notes
What LED? (1 or 2) <i>1</i>	
What interval (in msec)? <i>2000</i>	LED1 starts blinking at an interval of 2000ms on and 2000ms off. LED2 is unaffected.
What LED? (1 or 2) <i>2</i>	
What interval (in msec)? <i>1000</i>	LED2 starts blinking at an interval of 1000ms on and 1000ms off. LED1 is unaffected.
What LED? (1 or 2)	Waiting for next LED# interval pair

TESTING AND VALIDATION REQUIREMENTS:

Table 2. Testing and Validation Requirements

T.0	All testing and validation must be done on the testbed illustrated in Figure 1.
T.1	The dialog above (or similar) must work as shown
T.1A	The blink rate of an LED must be able to be set without interfering with the blinking of the other LED
T.1.1	Setting of the blink rate (and LED#) must be through the IDE serial monitor
T.2	The blink rate of the LED being set must not change until the input is complete
T.3	The user must specify the blink rate in milliseconds per blink
T.4	The blink rate specified by the user must be correctly reflected on the testbed LEDs.
T.4.1	The blink rate specified by the user must be reflected on the LED specified by the user
T.5.	The setting of an LED's blink rate must be able to be repeated at least 5 times
T.5.1.	Successively for the same LED
T.5.2.	Alternately between the different LEDs

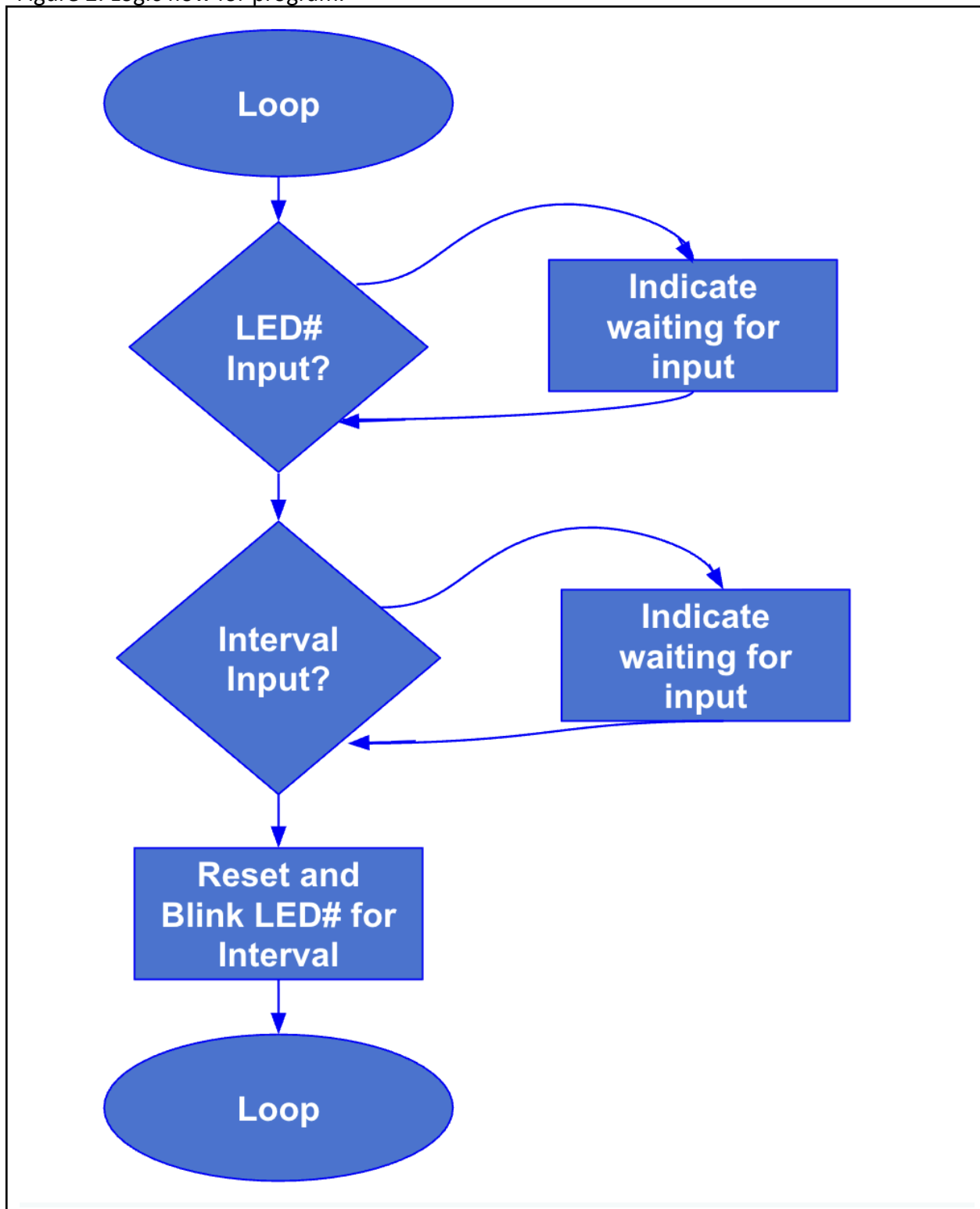
TESTING RESULTS:

Table 3. Results of tests

Test performed	Results
T.1.1	Satisfied. See video of test.
T.1	Satisfied
T.2	Satisfied. See video of test.
T.3	Satisfied
T.4	Satisfied
T.4.1	Satisfied. See video of test.
T.5.1	Satisfied. See video of test.
T.5.2	Satisfied. See video of test.

Appendix A. Logic Flow Chart

Figure 2. Logic flow for program.



APPENDIX B. DESIGN CODE

```
// PROJECT-CE

#define LED_PIN1 8
#define LED_PIN2 9

#include <Arduino.h>

unsigned long interval1 = 0;
unsigned long interval2 = 0;
unsigned long prevMillis1 = 0;
unsigned long prevMillis2 = 0;

bool state1 = LOW;
bool state2 = LOW;

int step = 0;
int chosen_led = 0;

void TaskToggleLED1();
void TaskToggleLED2();
void TaskSerialInput();

typedef void (*TaskFunc)();
TaskFunc tasks[] = {
    TaskToggleLED1,
    TaskToggleLED2,
    TaskSerialInput,
};

const uint8_t NUM_TASKS = sizeof(tasks) / sizeof(TaskFunc);

void setup() {
    Serial.begin(9600);
    pinMode(LED_PIN1, OUTPUT);
    pinMode(LED_PIN2, OUTPUT);
    digitalWrite(LED_PIN1, LOW);
    digitalWrite(LED_PIN2, LOW);
    Serial.println("What LED? (1 or 2)");
}

void loop() {
    for (uint8_t i = 0; i < NUM_TASKS; ++i) {
```



```

    tasks[i]();
}

}

void TaskToggleLED1() {
    unsigned long now = millis();
    if (interval1 > 0 && (now - prevMillis1 >= interval1)) {
        prevMillis1 = now;
        state1 = !state1;
        digitalWrite(LED_PIN1, state1 ? HIGH : LOW);
    }
}

void TaskToggleLED2() {
    unsigned long now = millis();
    if (interval2 > 0 && (now - prevMillis2 >= interval2)) {
        prevMillis2 = now;
        state2 = !state2;
        digitalWrite(LED_PIN2, state2 ? HIGH : LOW);
    }
}

void flushSerialInput() {
    while (Serial.available()) Serial.read();
}

void TaskSerialInput() {
    if (Serial.available()) {
        if (step == 0) {
            // Read chosen LED number
            int val = Serial.parseInt();
            flushSerialInput();
            if (val == 1 || val == 2) {
                chosen_led = val;
                step = 1;
                Serial.println("What interval (in msec)?");
            } else {
                Serial.println("Invalid LED. Please enter 1 or 2:");
            }
        } else if (step == 1) {
            long val = Serial.parseInt();

```

```
flushSerialInput();
if (val > 0) {
    if (chosen_led == 1) {
        interval1 = (unsigned long) val;
        prevMillis1 = millis();
    } else {
        interval2 = (unsigned long) val;
        prevMillis2 = millis();
    }
    Serial.print("LED ");
    Serial.print(chosen_led);
    Serial.print(" interval set to ");
    Serial.print(val);
    Serial.println(" ms");
    step = 0;
    Serial.println("What LED? (1 or 2)");
}
}
}
}
```

APPENDIX C. Video

Link to video in Google Drive:

<https://drive.google.com/file/d/18qfU-JEhk7RyHqM0Iy-k9SCxEjCGCsdG/view?usp=sharing>

END OF DOCUMENT